



南開大學
Nankai University

计算机学院
软件工程课程测试报告

滑动窗口最大值测试报告

姓名：林晖鹏
学号：2312966
专业：计算机科学与技术

2026 年 6 月 5 日

目录

1	题目与问题定义	3
2	算法设计与实现思路	3
2.1	功能需求	3
2.2	输入输出约定	3
2.3	非法输入处理约定	3
2.4	核心代码片段	3
3	编程规范说明	4
3.1	代码分析	5
4	可扩展性分析	6
4.1	现有扩展性	6
4.2	从数据规模角度的扩展性	6
4.3	从需求变化角度的扩展性	6
4.4	从用户使用角度的扩展性	6
4.5	从软件构建角度的扩展性	6
5	设计原则分析	7
5.1	单一职责原则	7
5.2	开放-封闭原则	7
6	错误与异常处理	7
7	算法复杂度分析	8
7.1	朴素解法	8
7.2	优化解法	9
7.3	两种解法复杂度对比	9
8	性能分析与优化	9
8.1	优化前的实现思路	10
8.2	性能分析	10
8.3	代码优化	11
8.4	有效性分析	11
8.5	优化结果分析	11
9	单元测试设计	12
9.1	黑盒测试设计	12
9.2	白盒测试设计	12
9.3	测试用例表	13
10	测试覆盖率与通过率	14
11	缺陷报告	15

12 总结

15

1 题目与问题定义

要求实现滑动窗口最大值算法。给定整数数组 *nums* 和窗口大小 *k*，窗口从数组左侧向右侧逐步滑动，每次移动一位。对于每一个窗口，要求出窗口范围内的最大值，并将这些最大值组成结果数组。

2 算法设计与实现思路

本报告实现了两种求解方式。第一种是朴素解法：对每个窗口直接求最大值，用于正确性校验和性能对比。第二种是优化解法：使用单调队列维护当前窗口中的候选最大值下标，将整体时间复杂度降到 $O(n)$ 。

代码结构采用职责分离的方式组织。输入校验由 `validate_inputs` 独立完成，朴素解法由 `max_sliding_window_bruteforce` 实现，优化解法由 `max_sliding_window` 实现。这样的划分便于测试、维护和后续扩展。

2.1 功能需求

程序需要完成以下功能：接收整数数组 *nums* 和窗口大小 *k*，计算滑动窗口每次移动后的窗口最大值，返回由所有窗口最大值组成的结果列表，并对非法输入给出明确异常。

2.2 输入输出约定

主接口定义为 `max_sliding_window(nums, k)`，另提供 `max_sliding_window_bruteforce(nums, k)` 作为朴素对照实现。输入输出约定如下：

- 输入： `nums: list[int]`, `k: int`
- 输出： `list[int]`

2.3 非法输入处理约定

程序对非法输入采用显式校验，规则如下：

- 当 *nums* 不是列表时，抛出 `TypeError`
- 当 *nums* 中存在非整数元素时，抛出 `TypeError`
- 当 *k* 不是整数时，抛出 `TypeError`
- 当 $k \leq 0$ 时，抛出 `ValueError`
- 当 $k > \text{len}(\text{nums})$ 时，抛出 `ValueError`
- 当 *nums* 为空列表时，抛出 `ValueError`

2.4 核心代码片段

输入校验

```
1 def validate_inputs(nums, k):
2     # 先在入口处统一校验输入，避免错误扩散到核心算法内部。
3     if not isinstance(nums, list):
4         raise TypeError("nums必须是整数列表")
```

```
5     if not isinstance(k, int):
6         raise TypeError("k必须是整数")
7     if not nums:
8         raise ValueError("nums不能为空")
9     if any(not isinstance(value, int) for value in nums):
10        raise TypeError("nums中的元素必须全部为整数")
11    if k <= 0:
12        raise ValueError("k必须大于0")
13    if k > len(nums):
14        raise ValueError("k不能大于nums的长度")
```

朴素解法核心循环

```
1 def max_sliding_window_bruteforce(nums, k):
2     validate_inputs(nums, k)
3     result = []
4
5     # 保留朴素解法是为了作为正确性对照和性能基线。
6     for start in range(len(nums) - k + 1):
7         result.append(max(nums[start : start + k]))
8
9     return result
```

优化解法核心循环

```
1 def max_sliding_window(nums, k):
2     validate_inputs(nums, k)
3     # 队列保存候选最大值的下标，便于同时判断是否滑出窗口。
4     window_indexes = deque()
5     result = []
6
7     for index, value in enumerate(nums):
8         # 队首一旦离开窗口范围，就不可能再成为后续窗口最大值。
9         if window_indexes and window_indexes[0] <= index - k:
10            window_indexes.popleft()
11
12        # 进入队列前移除所有更小元素，保证队首始终对应当前最大值。
13        while window_indexes and nums[window_indexes[-1]] <= value:
14            window_indexes.pop()
15        window_indexes.append(index)
16
17        if index >= k - 1:
18            result.append(nums[window_indexes[0]])
19
20    return result
```

3 编程规范说明

本代码遵守 Python 编程规范，主要参考 PEP 8，同时结合课程中关于命名清晰、注释适度、模块化设计和可维护性的要求进行实现。

实现中重点遵守了以下 PEP 8 相关规范：

- 模块、函数和变量采用清晰且一致的命名方式，例如 `validate_inputs`、`max_sliding_window`
- `import` 语句统一放在文件顶部，避免在业务逻辑中间插入导入语句
- 函数之间保留合理空行，使代码结构层次清楚
- 使用模块级文档字符串和函数文档字符串说明功能
- 控制注释数量，不用注释重复解释代码表面含义，而只解释关键设计意图
- 保持代码结构简洁，避免炫技写法和难以维护的复杂表达

3.1 代码分析

使用 `pylint` 对代码进行静态分析。检查范围不仅包括代码风格，还包括潜在错误、导入关系、命名规范、文档字符串、异常处理方式以及是否存在不符合最佳实践的写法。检查对象包括 `src/sliding_window.py`、`tests/test_sliding_window.py` 和 `analysis/benchmark.py`。

静态分析主要覆盖了以下内容：

- 模块是否能够被正确导入
- 是否缺少模块级或函数级文档字符串
- 变量、函数和模块命名是否符合规范
- 是否存在未使用变量或无意义导入
- 是否存在不合理的异常处理方式
- 导入顺序是否规范
- 代码结构是否容易触发常见的重构类告警

使用的检查命令为：

pylint 检查命令

```
1 PYLINTHOME='/private/tmp/pylint-assignment' python3 -m pylint \  
2 src/sliding_window.py \  
3 tests/test_sliding_window.py \  
4 analysis/benchmark.py
```

最终运行结果如下：

pylint 输出结果

```
1 -----  
2 Your code has been rated at 10.00/10
```

`pylint` 最终评分为 **10.00/10**。这个分数并不表示代码绝对完美，而是说明在 `pylint` 当前规则下，这几个文件已经没有剩余的错误、警告或风格违规项，因此得到了满分。

4 可扩展性分析

本代码在设计时考虑了扩展性，而且这种扩展性不仅体现在算法层面，也体现在数据规模、需求变化、用户使用方式和软件构建方式上。

4.1 现有扩展性

当前代码中已经体现出的扩展性主要包括：

- 输入校验逻辑独立，可被不同算法实现复用
- 朴素解法与优化解法分离，便于新增更多策略
- 对外保留统一功能接口，后续可扩展滑动窗口最小值、平均值或返回最大值位置等功能

当前版本已经从“只写一个题解函数”扩展成了“输入校验 + 朴素对照实现 + 优化实现 + 测试 + 性能分析”的小型工程结构。

4.2 从数据规模角度的扩展性

目前已经考虑了不同数据规模下的扩展性。朴素解法适合小规模数据校验正确性，优化解法适合更大规模数据处理。保留两种实现后，可以根据数据规模和用途选择不同策略。

未来如果数据规模继续增大，还可以进一步扩展：

- 增加批量性能测试脚本
- 增加更系统的性能基准工具
- 针对超大输入考虑内存占用和流式处理方式

4.3 从需求变化角度的扩展性

当前题目只要求返回窗口最大值，但代码结构已经为需求变化预留了空间。未来可以继续扩展：

- 滑动窗口最小值
- 滑动窗口平均值或中位数
- 返回最大值及其下标
- 支持不同窗口统计策略的统一接口

由于输入校验和不同算法实现已经拆分，新增这些能力时不需要推翻当前主体结构。

4.4 从用户使用角度的扩展性

从用户角度看，当前程序已经具备清晰的函数接口，便于其他模块直接导入调用，也便于测试代码复用。未来还可以继续扩展为命令行调用、文件输入输出以及更明确的运行日志和错误提示。

4.5 从软件构建角度的扩展性

本实验已经将源代码、测试、分析脚本、测试数据和报告分目录存放，这种组织方式本身就是可扩展性的体现。后续如果继续增加模块，不需要把所有内容堆在一个文件里，而是可以按职责继续扩展目录结构。

例如，未来还可以进一步加入：

- cli/ 或主程序入口文件
- 更多测试模块
- 自动化脚本
- 更完整的结果输出文件

5 设计原则分析

5.1 单一职责原则

单一职责原则的含义是：一个模块、类或函数应该只负责一项明确的职责。这样做的好处是，当需求发生变化时，变化会尽量被限制在单一位置，不会牵连太多无关代码，从而提高代码的可维护性和可测试性。

`validate_inputs` 只负责输入合法性检查，`max_sliding_window_bruteforce` 只负责朴素求解，`max_sliding_window` 只负责单调队列优化求解，测试文件只负责验证程序行为。各模块职责单一，彼此边界清楚。

5.2 开放-封闭原则

开放-封闭原则的含义是：软件实体应当对扩展开放、对修改封闭。也就是说，当需要增加新功能时，应尽量通过新增代码来完成，而不是频繁修改已有核心代码。这样可以降低引入回归错误的风险，并使系统结构更加稳定。

当前程序结构允许在不破坏现有调用方式的前提下增加新的滑动窗口实现策略，因此对扩展开放，对既有接口尽量保持稳定。这一结构也有助于减少后续修改对已有代码的影响范围。

6 错误与异常处理

错误与异常处理是本实验中的一个重点。滑动窗口最大值的核心算法本身并不复杂，但如果输入不合法、边界条件不完整，程序就可能在深层逻辑中才暴露错误。这种错误虽然也会报出来，但报错位置通常不明确，不利于定位问题，也不利于后续维护。

实验采用的处理思路是：在算法入口处统一完成输入合法性检查，把“正常计算逻辑”和“异常处理逻辑”分离。这样做有两个直接好处：

1. 核心算法函数内部只需要关注正确输入下的求解过程，逻辑更清晰
2. 错误能够在最靠近入口的位置被发现，异常原因也更明确

程序显式处理了以下错误情况：

- `nums` 不是列表
- `nums` 中含非整数元素
- `k` 不是整数
- `k <= 0`
- `k > len(nums)`

- `nums` 为空列表

对于这些错误，程序没有使用笼统的报错方式，而是按错误性质进行细分：

1. 当输入类型不符合要求时，例如 `nums` 不是列表、`nums` 中含非整数元素、`k` 不是整数，程序抛出 `TypeError`
2. 当输入类型正确但数值范围不合法时，例如 `k <= 0`、`k > len(nums)`、`nums` 为空列表，程序抛出 `ValueError`

这种区分方式的意义在于：`TypeError` 反映的是传入数据的类型格式不对，而 `ValueError` 反映的是传入数据取值不合法。这样不仅更符合 Python 的异常使用习惯，也使测试用例和后续维护时的定位更加直接。

此外，本代码没有使用裸 `except` 来笼统吞掉所有异常。这是因为裸 `except` 容易掩盖真实错误来源，也会降低调试效率。当前实现选择在输入层面做显式校验，并让异常按照明确类型抛出，这样既满足“针对具体异常类型处理”的要求，也保证了代码行为可预测。

从测试结果来看，异常路径已经通过单元测试覆盖，包括：

- 非列表输入
- `None` 输入
- 非整数窗口大小
- 非法元素类型
- 空列表
- `k = 0`
- `k < 0`
- `k > len(nums)`

这部分实现不仅考虑了错误与异常处理，而且已经通过测试验证了相关异常行为是否符合预期。

7 算法复杂度分析

算法	时间复杂度	空间复杂度
朴素解法	$O(nk)$	$O(1)$
单调队列优化解法	$O(n)$	$O(k)$

表 1: 两种算法复杂度对比

7.1 朴素解法

朴素解法的思路是：当窗口每向右移动一次，就重新扫描当前窗口中的 k 个元素，并调用 `max()` 求出该窗口的最大值。

设输入数组长度为 n ，窗口大小为 k ，则一共需要处理的窗口个数为：

$$n - k + 1$$

对于每一个窗口，朴素解法都需要重新遍历这 k 个元素求最大值，因此单个窗口的代价为 $O(k)$ 。于是总时间复杂度为：

$$O((n - k + 1) \cdot k)$$

只保留主导项后，可记为 $O(nk)$ 。

空间方面，朴素解法在求每个窗口最大值时没有额外维护复杂的数据结构，除输出结果外只使用了常数个辅助变量，因此空间复杂度为 $O(1)$ （不计输出结果）。

朴素解法的主要问题在于：相邻窗口之间存在大量重叠元素，但程序仍然会反复扫描这些元素，因此重复计算较多。

7.2 优化解法

优化解法使用单调队列维护“当前窗口内仍然有资格成为最大值的元素下标”。队列中的下标对应值始终保持单调递减，因此队首元素始终对应当前窗口最大值。

在遍历数组过程中，每个元素会经历以下操作：

1. 最多入队一次；
2. 最多因“滑出窗口”从队首移出一次；
3. 最多因“比新元素小”从队尾移出一次。

也就是说，虽然代码中存在 `while` 循环，但从整体上看，每个元素不会被无限次处理，而是在整个算法执行过程中至多进入队列一次、离开队列一次。因此所有入队、出队操作的总次数与 n 成正比。

于是优化解法的总时间复杂度为 $O(n)$ 。

空间方面，单调队列中最多只会保存当前窗口范围内的若干下标，队列长度不会超过窗口大小 k ，因此空间复杂度为 $O(k)$ 。

这种复杂度改进的本质在于：程序不再对每个窗口重复扫描全部元素，而是通过单调队列保留“可能成为最大值”的候选元素，从而把重复计算压缩到最低。

7.3 两种解法复杂度对比

两种解法的复杂度差异可以概括如下：

- 朴素解法：每个窗口独立求最大值，重复扫描窗口内部元素，时间复杂度为 $O(nk)$ ；
- 优化解法：用单调队列复用相邻窗口之间的信息，时间复杂度降为 $O(n)$ 。

当 k 较大或者 n 持续增大时， $O(nk)$ 和 $O(n)$ 之间的差距会迅速扩大，这也是后续性能测试中优化解法明显更快的根本原因。

8 性能分析与优化

这里使用 `analysis/benchmark.py` 和 Python 标准库中的性能分析工具 `cProfile`（即 `profile` 的 C 实现版本）对两种算法进行性能分析。

测试参数为三组数据规模： $n = 1000$ 、 5000 、 10000 ，窗口大小固定为 $k = 100$ 。一次测试结果如表 2 所示。

数据规模	朴素解法耗时（秒）	优化解法耗时（秒）
1000	0.002354	0.000796
5000	0.013155	0.002286
10000	0.025859	0.004760

表 2: 两种算法性能对比

结果表明，优化解法在不同数据规模下都明显快于朴素解法。进一步的 **cProfile** 结果显示，在 $n = 10000$ 、 $k = 100$ 的测试中，优化解法共发生 49901 次函数调用，总耗时约为 0.028 秒，主要时间开销集中在表 3 所示的几个核心函数与操作上。

函数/操作	调用次数	累计耗时（秒）	说明
<code>max_sliding_window</code>	1	0.028	核心求解函数
<code>validate_inputs</code>	1	0.008	输入校验
<code>builtins.any</code>	1	0.008	非法元素检查
<code><genexpr></code>	10001	0.006	输入合法性生成器表达式

表 3: cProfile 关键结果

8.1 优化前的实现思路

优化前采用的是朴素解法。它的基本思路是：每当窗口向右滑动一次，就重新截取当前窗口中的 k 个元素，并直接调用 `max()` 求最大值。

这种方法实现简单，也便于作为正确性对照，但它存在一个明显问题：窗口之间高度重叠，相邻两个窗口只相差一个移出元素和一个移入元素，可朴素解法仍然会把整个窗口重新扫描一遍，因此产生了大量重复计算。

8.2 性能分析

这里用统一脚本对朴素解法和优化解法做对照测试。性能分析过程分为两步：

1. 使用 `analysis/benchmark.py` 比较两种算法在不同数据规模下的总耗时。
2. 使用 **cProfile** 对优化解法做函数级分析，观察主要耗时集中在哪些函数和操作上。

这种做法的目的不是只得到一个“快还是慢”的结论，而是让优化具有证据基础：先确认哪种实现更快，再分析快慢差异来自哪里。

由于本实验程序规模较小，核心逻辑主要集中在少数函数中，因此本次性能分析的重点放在不同算法实现的整体耗时对比上，**cProfile/profile** 的作用主要是辅助确认主要时间开销确实集中在核心求解函数，而不是用于复杂系统中的细粒度模块瓶颈定位。

如果需要进一步做语句级或逐行级分析，也可以使用 `line_profiler` 等更细粒度的工具作为补充。本实验仍以 Python 标准库中的 `profile/cProfile` 作为主要分析工具。

8.3 代码优化

本次优化的核心不是调整语法细节，而是替换算法和数据结构。

朴素解法的主要问题是：每次窗口移动后都重新扫描窗口内部全部元素，因此时间复杂度为 $O(nk)$ 。要优化这个过程，就需要避免重复扫描窗口内部的旧元素。

采用的优化方法是单调队列。具体思路如下：

- 队列中不直接保存元素值，而是保存元素下标
- 队列中的下标对应值保持单调递减
- 当新元素进入窗口时，移除队尾所有比它小的元素，因为这些元素在当前窗口和后续窗口中都不可能再成为最大值
- 当窗口左端滑动时，如果队首下标已经不在窗口范围内，就将其移除
- 这样，队首下标始终对应当前窗口最大值

这种优化的关键在于：每个元素最多只会进入队列一次、移出队列一次，因此整体时间复杂度可以降到 $O(n)$ 。

8.4 有效性分析

- 在朴素解法中，同一个元素可能会被多个相邻窗口反复比较
- 在单调队列解法中，一个元素一旦被判定不可能再成为最大值，就会被及时移除，不再参与后续比较

也就是说，优化的本质是利用数据结构保存“仍然有资格成为最大值的候选元素”，而不是每次都从头计算。

8.5 优化结果分析

从测试结果可以看出，随着数据规模增大，优化解法和朴素解法之间的差距会越来越明显。这说明当前优化并不是偶然的常数项改进，而是复杂度层面的改进。

进一步的 cProfile 结果显示，在数据规模 $n = 10000$ 、窗口大小 $k = 100$ 的测试中，优化解法共发生 49901 次函数调用，总耗时约为 0.028 秒。其中，主要时间开销集中在核心求解函数 `max_sliding_window`、输入校验函数 `validate_inputs` 以及输入合法性检查中的生成器表达式上；而 `deque` 的 `append`、`pop` 和 `popleft` 等操作虽然调用频繁，但单次耗时较低。

为了进一步观察核心函数内部的耗时分布，这里又使用 `line_profiler` 对 `max_sliding_window` 做了逐行分析。

结果表明，`validate_inputs(nums, k)` 约占 12.1%，维护单调队列的 `while window_indexes and nums[window_indexes[-1]] <= value` 约占 21.8%，将窗口最大值写入结果的 `result.append(nums[window_indexes[0]])` 约占 14.0%。这说明优化解法的主要成本确实集中在队列维护、窗口边界判断和结果输出这些核心步骤上，而不是分散在无关辅助逻辑中。

性能优化的关键不在于语法层面的微调，而在于通过更合适的数据结构减少重复计算。

9 单元测试设计

测试使用 `unittest` 编写，并结合黑盒测试与白盒测试思想设计。黑盒测试部分依据题目需求覆盖正常输入、边界输入和异常输入。白盒测试部分针对关键分支和错误处理路径补充测试用例。

从具体方法上看，本次测试设计主要包含以下四类：

1. 有效等价类：为正常、合法输入设计代表性样例，如标准示例、严格递增、严格递减、全相同元素、含负数等，用于验证程序是否实现了题目要求的基本功能。
2. 无效等价类：对 `nums=None`、非列表输入、元素类型非法、`k` 非整数等不合理输入单独设计测试，用于验证程序的容错与异常处理能力。
3. 边界值与健壮性测试：重点测试 `k=1`、`k=len(nums)`、单元素数组、`k=0`、`k<0`、`k>len(nums)`、空列表等边界和越界场景。这里既覆盖了边界值分析，也体现了健壮性测试中“略小于最小值、略大于最大值”的思想。
4. 白盒补充测试：针对输入校验分支、窗口左边界滑出、窗口右边界进入、原最大值滑出后更新等关键分支与关键路径补充测试，以增强对内部逻辑的覆盖。

当前测试覆盖以下场景：

- 标准功能测试
- 边界值测试
- 异常输入测试
- 严格递增、严格递减、重复值、负数等特征数据测试
- 朴素解法与优化解法一致性测试

9.1 黑盒测试设计

黑盒测试不依赖程序内部实现细节，而是依据题目需求、输入约束和输出约定设计测试用例。本实验中的黑盒测试主要包含以下三类：

1. 有效等价类：为正常、合法输入设计代表性样例，如标准示例、严格递增、严格递减、全相同元素、含负数等，用于验证程序是否实现了题目要求的基本功能。
2. 无效等价类：对 `nums=None`、非列表输入、元素类型非法、`k` 非整数等不合理输入单独设计测试，用于验证程序的容错与异常处理能力。
3. 边界值与健壮性测试：重点测试 `k=1`、`k=len(nums)`、单元素数组、`k=0`、`k<0`、`k>len(nums)`、空列表等边界和越界场景。这里既覆盖了边界值分析，也体现了健壮性测试中“略小于最小值、略大于最大值”的思想。

9.2 白盒测试设计

白盒测试关注程序内部逻辑结构，强调关键分支、关键路径和错误处理路径是否被测试触达。本实验中的白盒测试主要包括：

1. 输入校验分支测试：验证 `validate_inputs` 中的类型判断、空列表判断和窗口范围判断是否都能正确进入对应分支。
2. 窗口边界变化测试：验证窗口左边界滑出和右边界进入时，队列状态更新是否正确。
3. 最大值更新路径测试：验证原最大值滑出窗口后，程序能否正确选择新的窗口最大值。
4. 一致性互验测试：将朴素解法与优化解法在同一输入上的结果逐一比较，用于辅助验证主算法内部逻辑没有偏差。
5. `mock` 隔离依赖测试：在文本样例读取测试中，使用 `unittest.mock` 模拟 `Path.read_text` 的返回值，隔离真实文件系统依赖，验证样例解析逻辑本身是否正确。

9.3 测试用例表

编号	设计方法	测试场景	输入	预期结果
TC01	有效等价类	标准示例	<code>nums=[1,3,-1,-3,5,3,6,7]</code> , <code>k=3</code>	<code>[3,3,5,5,6,7]</code>
TC02	边界值	窗口大小为 1	<code>nums=[4,-2,9]</code> , <code>k=1</code>	<code>[4,-2,9]</code>
TC03	边界值	窗口覆盖整个数组	<code>nums=[2,7,1,5]</code> , <code>k=4</code>	<code>[7]</code>
TC04	边界值	单元素数组	<code>nums=[42]</code> , <code>k=1</code>	<code>[42]</code>
TC05	有效等价类	严格递增数组	<code>nums=[1,2,3,4,5]</code> , <code>k=3</code>	<code>[3,4,5]</code>
TC06	有效等价类	严格递减数组	<code>nums=[5,4,3,2,1]</code> , <code>k=2</code>	<code>[5,4,3,2]</code>
TC07	有效等价类	全相同元素	<code>nums=[6,6,6,6]</code> , <code>k=3</code>	<code>[6,6]</code>
TC08	有效等价类	重复最大值	<code>nums=[4,4,4,2,2]</code> , <code>k=2</code>	<code>[4,4,4,2]</code>
TC09	有效等价类	含负数数组	<code>nums=[-7,-8,-6,-5,-9]</code> , <code>k=2</code>	<code>[-7,-6,-5,-5]</code>
TC10	白盒补充	最大值位于左边界	<code>nums=[9,3,2,1]</code> , <code>k=3</code>	<code>[9,3]</code>
TC11	白盒补充	最大值位于右边界	<code>nums=[1,2,9,3]</code> , <code>k=3</code>	<code>[9,9]</code>
TC12	健壮性测试	<code>k=0</code>	<code>nums=[1,2,3]</code> , <code>k=0</code>	抛出 <code>ValueError</code>
TC13	健壮性测试	<code>k<0</code>	<code>nums=[1,2,3]</code> , <code>k=-1</code>	抛出 <code>ValueError</code>
TC14	健壮性测试	<code>k>len(nums)</code>	<code>nums=[1,2]</code> , <code>k=3</code>	抛出 <code>ValueError</code>
TC15	无效等价类	空列表输入	<code>nums=[]</code> , <code>k=1</code>	抛出 <code>ValueError</code>
TC16	无效等价类	<code>nums=None</code>	<code>nums=None</code> , <code>k=2</code>	抛出 <code>TypeError</code>
TC17	无效等价类	非列表输入	<code>nums="123"</code> , <code>k=2</code>	抛出 <code>TypeError</code>

编号	设计方法	测试场景	输入	预期结果
TC18	无效等价类	k 非整数	nums=[1,2,3], k=1.5	抛出 TypeError
TC19	无效等价类	数组中含非法元素	nums=[1,"2",3], k=2	抛出 TypeError
TC20	有效等价类	朴素解法标准示例	nums=[1,3,-1,-3,5,3,6,7], k=3	[3,3,5,5,6,7]
TC21	白盒补充	两种实现一致性	nums=[9,1,3,7,2,6,8,4], k=4	两种实现结果一致
TC22	白盒补充	原最大值滑出窗口后更新最大值	nums=[8,1,1,7], k=3	[8,7]

10 测试覆盖率与通过率

在说明本次覆盖率结果之前，先对测试覆盖标准中的几个基本概念做出说明：

- 测试需求：测试需求是软件制品的一个特定元素，测试用例必须满足或覆盖这个特定元素。在本实验中，测试需求可以理解为某个输入类别、某条异常处理规则、某个关键分支或某种算法行为。
- 覆盖标准：覆盖标准是一条规则，或者是将测试需求施加在一个测试集上的一组规则。本实验选择“行覆盖率”作为覆盖标准中的主要度量指标。
- 测试覆盖：给定覆盖标准 C 和测试需求集合 TR ，若测试集合 T 中对 TR 的每一条测试需求都至少存在一个测试用例进行满足，则认为 T 满足该覆盖标准。
- 覆盖程度：覆盖程度是测试集合 T 满足的测试需求数占测试需求总数的比例。在本实验中，使用 `trace` 统计得到的行覆盖率就是覆盖程度的一种具体体现。

本实验通过 `unittest` 执行了 22 条核心单元测试，测试通过率为 $22/22 = 100\%$ 。此外，在提交代码目录中还补充了 2 条辅助测试：1 条文本样例回归测试，用于验证 `in*.txt / out*.txt` 测试数据能够被程序正确读取和执行；1 条基于 `unittest.mock` 的测试，用于隔离文件读取依赖并验证样例解析逻辑。

在 `code/` 目录下执行的命令如下：

```
python3 -m unittest tests.test_sliding_window tests.test_txt_cases -v
```

对应结果表明当前共运行 24 条测试，全部通过：

```
.....
-----
Ran 24 tests in 0.028s

OK
```

本实验选择行覆盖率（line coverage）作为覆盖指标，并使用 Python 标准库 `trace` 对测试覆盖情况进行了统计。之所以选择该指标，是因为本实验程序规模较小、核心逻辑集中，使用行覆盖率可以较直观地说明核心代码是否被测试触达。统计结果显示，代码覆盖率为 100%。需要说明的是，覆盖率

只能说明“哪些代码被执行过”，不能单独代表测试质量，因此仍需结合等价类、边界值和白盒分支与关键路径补充测试来综合判断测试是否充分。

除标准库 `trace` 外，本实验还补充使用了 `coverage.py` 做覆盖率实验。在 `code/` 目录下执行的命令如下：

```
1 PYTHONPATH=/private/tmp/coverage_pkg python3 -m coverage run --source=src -m unittest
  tests.test_sliding_window tests.test_txt_cases
2 PYTHONPATH=/private/tmp/coverage_pkg python3 -m coverage report -m
```

对应输出如下：

Name	Stmts	Miss	Cover	Missing
src/__init__.py	0	0	100%	
src/sliding_window.py	33	0	100%	
TOTAL	33	0	100%	

可见 `coverage.py` 统计结果同样显示 `src/sliding_window.py` 的覆盖率为 100%，并生成了 HTML 报告，文件位置为 `code/coverage_html/index.html`。该实验说明，在不同覆盖率工具下，当前核心源文件的覆盖结果是一致的。

11 缺陷报告

本轮单元测试中，未发现 `sliding_window.py` 中的被测代码缺陷。认为当前版本在本次实验范围内可以视为“未发现缺陷”，主要依据以下维度：

1. 正确性维度：当前 22 条核心单元测试全部通过，标准功能、边界条件和异常输入场景均已覆盖；另外补充的文本样例回归测试与 `mock` 辅助测试也已通过。
2. 互验维度：程序同时实现了朴素解法 `max_sliding_window_bruteforce` 和单调队列优化解法 `max_sliding_window`。在一致性测试中，两种独立实现对同一输入给出的结果完全一致，未发现策略差异。
3. 覆盖率维度：使用 Python 标准库 `trace` 统计得到 `src.sliding_window` 代码覆盖率为 100%，说明核心源文件中的可执行语句已经全部被测试触达，不存在明显的未执行死代码。
4. 性能分析维度：`cProfile` 与 `line_profiler` 结果均表明，主要时间开销集中在核心算法逻辑及必要输入校验上，没有发现异常的额外性能热点或无关开销。

同时也需要说明，“未发现缺陷”并不等价于“绝对不存在缺陷”。由于当前测试范围仍主要围绕课程作业要求展开，若后续扩展到更复杂输入类型、更大规模数据或其他使用场景，仍有必要继续补充测试与验证。

12 总结

本轮测试表明，当前程序在本次实验要求范围内运行结果正确，未发现功能性缺陷。该结论主要由以下证据支持：22/22 核心单元测试全部通过，补充的文本样例回归测试与 `mock` 辅助测试也通过，`src.sliding_window` 代码覆盖率为 100%，并且 `coverage.py` 的实验结果与此一致；朴

素解法与优化解法在一致性测试中结果完全一致，`pylint` 静态分析评分为 10.00/10，`cProfile` 与 `line_profiler` 的分析结果也未显示异常的无关性能开销。

需要说明的是，本结论成立的前提是测试范围仍以课程实验要求为主，主要覆盖正常输入、边界输入、异常输入以及典型性能场景。若后续扩展到更复杂输入类型、更大规模数据或其他应用场景，仍应继续补充测试样例和验证手段。